

LLM Component Schema Guide

This guide explains how to author and apply LLM Component Schemas to make design system components programmable, controllable, and AI-readable. Tokens are the DNA. Schema is the skeleton. Generative rules are the brain.

Public distribution edition. Designed for fast scanning, deep reference, and accessible reading.

Contents

1. [Why schema exists](#)
2. [Template locations](#)
3. [Choosing the right template](#)
4. [Schema anatomy \(blueprint\)](#)
5. [AI design system stack](#)
6. [Token layers \(Atomic DNA\)](#)
7. [Behavior and interaction](#)
8. [States, transitions, constraints](#)
9. [Accessibility mapping](#)
10. [Generative rules](#)
11. [Tailwind-specific schema entries](#)
12. [Implementation steps](#)
13. [Storybook integration](#)
14. [Validation checklist](#)

Why schema exists

Without schema, AI can only render pixels. With schema, AI can reason about behavior, constraints, and intent. A schema defines the component as a system of meaning, not just a visual output.

- Schema defines intent and structure, not just appearance.
- It standardizes behavior across variants and contexts.
- It provides a single source of truth for constraints.
- It enables automated validation and consistency checks.

Template locations

Use the templates located in:

- docs/LLM_COMPONENT_SCHEMA_TEMPLATE/LLM_COMPONENT_SCHEMA_TEMPLATE.json
- docs/LLM_COMPONENT_SCHEMA_TEMPLATE/LLM_COMPONENT_SCHEMA_TEMPLATE_TAILWINDCSS.json

Each component should include a schema.json file inside its component folder, for example:

```
nextjs-app/shared/components/Accordion/schema.json
```

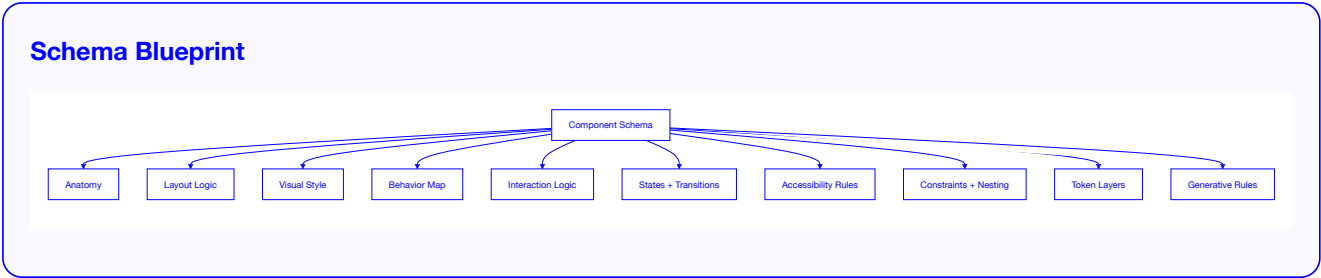
Choosing the right template

Choose the base template for token-driven or CSS-module components. Choose the Tailwind template when Tailwind utility classes define the visual system and state logic.

Template	Use when	Key additions
Base	Styles are token-driven, CSS modules, or inline styles.	Token mapping, behavior, constraints.
Tailwind	Tailwind utilities encode spacing, color, states, and layout.	Class maps, variant sources, safelist, class constraints.

Schema anatomy (blueprint)

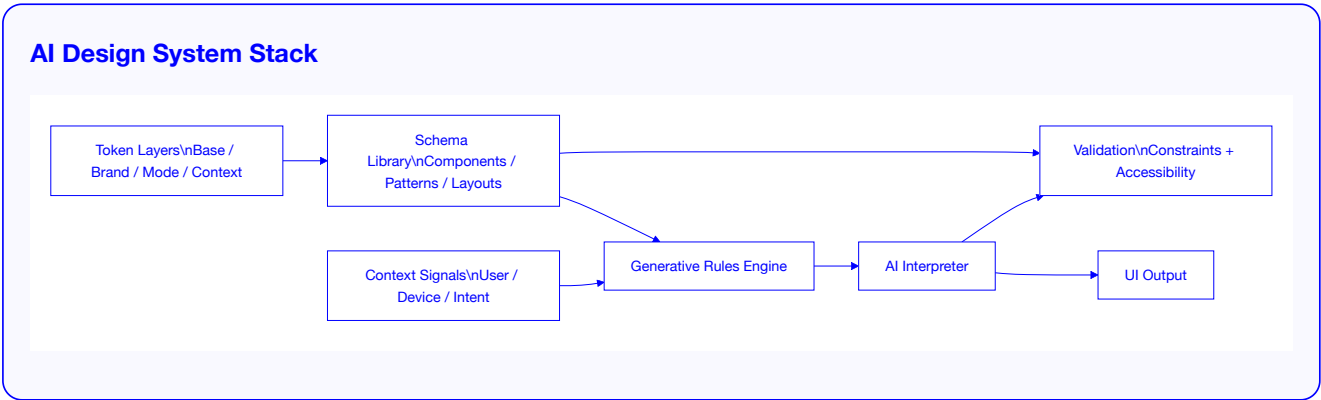
The schema is organized into predictable sections so AI can understand anatomy, layout, visual rules, behavior, and constraints.



A component is not a rectangle. It is an object with semantic roles, size rules, spacing constraints, interaction feedback, and behavior.

AI design system stack

Tokens define what is possible. Schema defines what is allowed. Rules define how components adapt to context. The interpreter enforces it.



Token layers (Atomic DNA)

Tokens are layered so AI can reason about intent and context. Base tokens define primitives, brand tokens define voice, mode tokens define theme, and contextual tokens adapt to needs such as accessibility.

Layer	Purpose	Examples
Base	Raw primitives and scale	token.color.gray.900, token.space.16
Brand	Identity and voice	brand.accent, brand.display
Mode	Theme-specific overrides	mode.dark.surface, mode.light.text
Contextual	Emotion, season, accessibility	context.a11y.highContrast, context.emotion.calm

Behavior and interaction

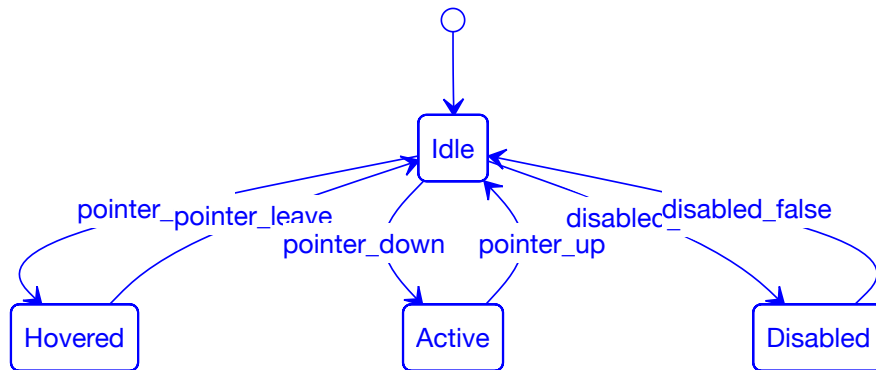
Behavior explains what the component does by default. Interaction logic describes triggers and feedback in every state.

```
"behavior": {
  "map": "Clicking a trigger opens a panel and closes others.",
  "stateModel": "uncontrolled",
  "rendering": "Panels render only when open.",
  "defaultState": "all closed"
},
"interaction": {
  "logic": [
    { "trigger": "onClick", "effect": "toggle panel" }
  ],
  "feedback": {
    "hover": "highlight trigger",
    "focus": "focus ring",
    "active": "press feedback"
  }
}
```

States, transitions, constraints

States describe a finite set of conditions. Transitions explain how the component moves between those states. Constraints define what is allowed.

State Transitions



```
"constraints": {
  "layout": ["Fixed height in compact mode."],
  "content": ["Label must be <= 2 lines."],
  "nesting": [
    "Card cannot contain another Card.",
    "Button cannot contain form elements."
  ]
}
```

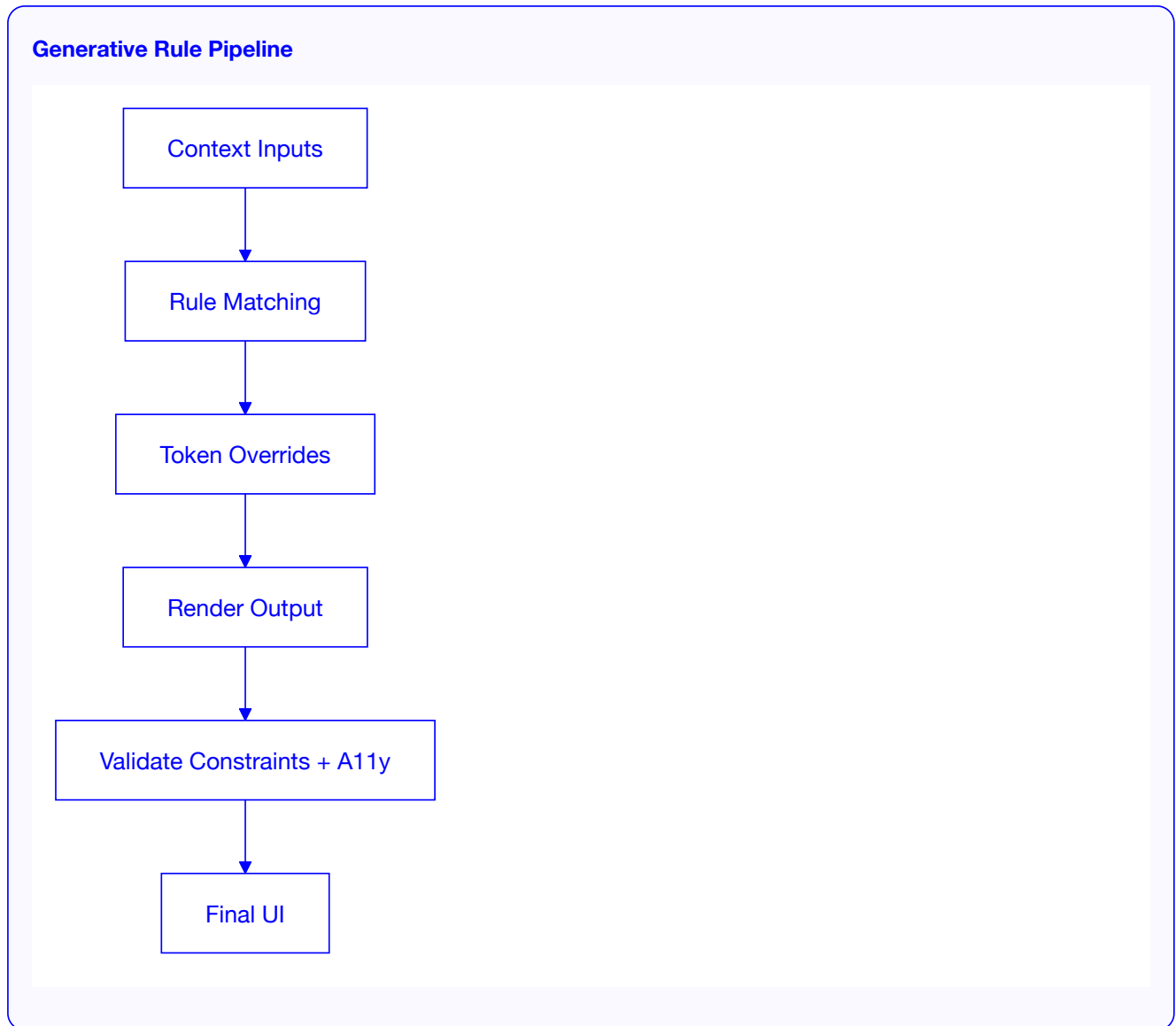
Accessibility mapping

Accessibility rules make semantics explicit and enforceable. Use roles, aria relationships, and keyboard behavior to define expected interaction.

```
"accessibility": {
  "roles": {
    "container": "div",
    "primary": "button",
    "panel": "region"
  },
  "aria": {
    "expanded": "true when open",
    "controls": "panel-id",
    "labelledBy": "trigger-id"
  },
  "keyboard": "Enter/Space toggles panel"
}
```

Generative rules

Generative rules transform context into token overrides. They are the bridge between intent and appearance.



```
"generativeRules": {
  "contextInputs": ["variant", "mode", "density"],
  "rules": [
    { "if": { "variant": "primary" }, "then": { "tokens": { "color": "brand.accent" } } },
    { "if": { "variant": "destructive" }, "then": { "tokens": { "color": "semantic.error" } } },
    { "if": { "mode": "dark" }, "then": { "tokens": { "surface": "neutral.surfaceElevated" } } }
  ]
}
```

Tailwind-specific schema entries

The Tailwind template adds structured class maps so AI understands how utility classes encode layout, states, and variants.

- **layout.classMap** encodes base layout utilities.
- **visualStyle.classMap** maps surface, radius, and typography utilities.
- **interaction.classMap** lists state and attribute variants.
- **states.classMap** documents class expectations by state.
- **tailwind.classComposition** defines how variants are assembled.
- **tailwind.safelist** ensures dynamic classes are preserved.

```
"interaction": {
  "classMap": {
    "hover": ["hover:bg-surface-hover"],
    "focus": ["focus-visible:ring-2", "focus-visible:ring-brand-500"],
    "dataState": ["data-[state=open]:bg-surface-2"]
  }
},
"tailwind": {
  "variantSources": ["hover", "focus-visible", "data-[state=open]"]
}
```

Implementation steps

1. Copy the correct template into the component folder as `schema.json`.
2. Fill in component metadata, anatomy, layout, visual style, and tokens.
3. Define behavior, interactions, states, and constraints.
4. Add accessibility roles and keyboard interactions.
5. Include generative rules for context-driven overrides.
6. For Tailwind: define class maps and variant sources explicitly.

Storybook integration

Import the schema into the component story and expose it via Storybook parameters. This allows humans and LLMs to verify the schema at the documentation layer.

```
import schema from "./schema.json";

const meta = {
  title: "Components/MyComponent",
  component: MyComponent,
  tags: ["autodocs"],
  parameters: {
    llm: { schema }
  }
};
```

Validation checklist

- Schema uses required sections with meaningful content.
- Behavior and interaction maps match actual component behavior.
- Constraints align with real layout limitations.
- Accessibility roles and keyboard rules are accurate.
- Generative rules map to tokens or Tailwind classes correctly.
- Storybook exposes schema for review and auditing.